

第 11 章 图论模型与算法

学习目标

- ☑ 掌握无根树的常用存储法和转化为有根树的方法
- ☑ 掌握由表达式构造表达式树的算法
- ☑ 掌握 Kruskal 算法及其正确性证明，并用并查集实现
- ☑ 掌握基于优先队列的 Dijkstra 算法实现
- ☑ 掌握基于 FIFO 队列的 Bellman-Ford 算法实现
- ☑ 掌握 Floyd 算法和传递闭包的求法
- ☑ 理解最大流问题的概念、流量的 3 个条件、残量网络的概念和求法
- ☑ 理解增广路定理与最小割最大流定理的证明方法，会实现 Edmonds-Karp 算法
- ☑ 理解最小费用最大流问题的概念，以及平行边和反向弧可能造成的问题
- ☑ 会实现基于 Bellman-Ford 的最小费用路算法
- ☑ 学会用网络流算法求解二分图最大基数匹配和最大权完美匹配
- ☑ 学会最小费用循环流的消圈算法

本章介绍一些常见的图论模型和算法，包括最小生成树、单源最短路、每对结点的最短路、最大流、最小费用最大流等。限于篇幅，很多算法都没有给出完整的正确性证明（很容易在其他参考资料中找到相关内容），但给出了简单、易懂的完整代码，方便读者参考。

11.1 再 谈 树

在第 6 章中，我们第一次接触到二叉树；后来，又接触到了其他树状结构，如解答树、BFS 树。本节将继续讨论“树”这一话题。

有 n 个顶点的树具有以下 3 个特点：连通、不含圈、恰好包含 $n-1$ 条边。有意思的是，具备上述 3 个特点中的任意两个，就可以推导出第 3 个，有兴趣的读者不妨试着证明一下。

11.1.1 无根树转有根树

输入一个 n 个结点的无根树的各条边，并指定一个根结点，要求把该树转化为有根树，输出各个结点的父结点编号。 $n \leq 10^6$ ，如图 11-1 所示。

【分析】

树是一种特殊的图，因此很容易想到用

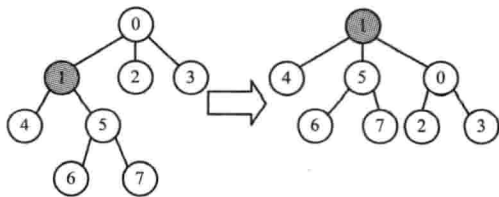


图 11-1 无根树转有根树

邻接矩阵表示。可惜, n 个结点的图对应的邻接矩阵要占用 n^2 个元素的空间, 开不下。怎么办呢? 用 `vector` 数组即可。由于 n 个结点的树只有 $n-1$ 条边, `vector` 数组实际占用的空间与 n 成正比。

```
vector<int> G[maxn];
void read_tree() {
    int u, v;
    scanf("%d", &n);
    for(int i = 0; i < n-1; i++) {
        scanf("%d%d", &u, &v);
        G[u].push_back(v);
        G[v].push_back(u);
    }
}
```

转化过程如下:

```
void dfs(int u, int fa) { //递归转化以 u 为根的子树, u 的父结点为 fa
    int d = G[u].size(); //结点 u 的相邻点个数
    for(int i = 0; i < d; i++) {
        int v = G[u][i]; //结点 u 的第 i 个相邻点 v
        if(v != fa) dfs(v, p[v] = u); //把 v 的父结点设为 u, 然后递归转化以 v 为根的子树
    }
}
```

主程序中设置 `p[root] = -1` (表示根结点的父结点不存在), 然后调用 `dfs(root, -1)` 即可。初学者最容易犯的错误之一就是忘记判断 v 是否和其父结点相等。如果忽略, 将引起无限递归。

11.1.2 表达式树

二叉树是表达式处理的常用工具。例如, $a+b*(c-d)-e*f$ 可以表示成如图 11-2 所示的二叉树。

其中, 每个非叶结点表示一个运算符, 左子树是第一个运算数对应的表达式, 而右子树则是第二个运算数对应的表达式。如何给一个表达式建立表达式树呢? 方法有很多, 这里只介绍一种: 找到“最后计算”的运算符 (它是整棵表达式树的根), 然后递归处理。下面是程序:

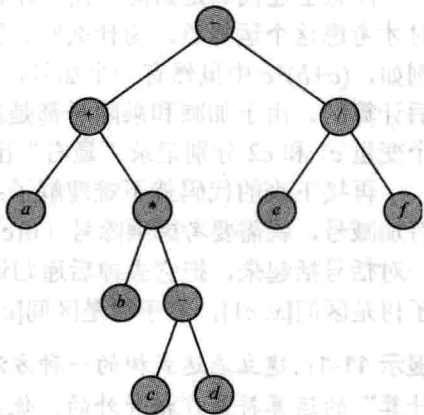


图 11-2 表达式树

```
const int maxn = 1000;
int lch[maxn], rch[maxn]; char op[maxn]; //每个结点的左右子结点编号和字符
int nc = 0; //结点数
```

```
int build_tree(char* s, int x, int y) {
    int i, c1=-1, c2=-1, p=0;
    int u;
    if(y-x == 1){ //仅一个字符，建立单独结点
        u = ++nc;
        lch[u] = rch[u] = 0; op[u] = s[x];
        return u;
    }
    for(i = x; i < y; i++) {
        switch(s[i]) {
            case '(': p++; break;
            case ')': p--; break;
            case '+': case '-': if(!p) c1=i; break;
            case '*': case '/': if(!p) c2=i; break;
        }
    }
    if(c1 < 0) c1 = c2; //找不到括号外的加减号，就用乘除号
    if(c1 < 0) return build_tree(s, x+1, y-1); //整个表达式被一对括号括起来
    u = ++nc;
    lch[u] = build_tree(s, x, c1);
    rch[u] = build_tree(s, c1+1, y);
    op[u] = s[c1];
    return u;
}
```

注意上述代码是如何寻找“最后一个运算符”的。代码里用了一个变量 p ，只有当 $p=0$ 时才考虑这个运算符。为什么呢？因为括号里的运算符一定不是最后计算的，应当忽略。例如， $(a+b)*c$ 中虽然有一个加号，但却是在括号里的，实际上比它优先级高的乘号才是最后计算的。由于加减和乘除号都是左结合的，最后一个运算符才是最后计算的，所以用两个变量 $c1$ 和 $c2$ 分别记录“最右”出现的加减号和乘除号。

再接下来的代码就不难理解了：如果括号外有加减号，它们肯定最后计算；但如果没加减号，就需要考虑乘除号（if($c1 < 0$) $c1 = c2$ ）；如果全都没有，说明整个表达式外面被一对括号括起来，把它去掉后递归调用。这样，就找到了最后计算的运算符 $s[c1]$ ，它的左子树是区间 $[x, c1]$ ，右子树是区间 $[c1+1, y]$ 。

提示 11-1：建立表达式树的一种方法是每次找到最后计算的运算符，然后递归建树。“最后计算”的运算符是在括号外的、优先级最低的运算符。如果有多个，根据结合性来选择：左结合的（如加、减、乘、除）选最右边；右结合的（如乘方）选最左边。根据规定，优先级相同的运算符的结合性总是相同。

例题 11-1 公共表达式消除 (Common Subexpression Elimination, ACM/ICPC NWERC 2009, UVa12219)

可以用表达式树来表示一个表达式。在本题中，运算符均为二元的，且运算符和运算

数均用 1~4 个小写字母表示。例如, $a(b(f(a,a),b(f(a,a),f)),f(b(f(a,a),b(f(a,a),f)),f))$ 可以表示为图 11-3 (a) 中形式。

用消除公共表达式的方法可以减少表达式树上的结点, 得到一个图, 如图 11-3 (b) 所示。左图有 21 个点, 而右图只有 7 个点。其表示方法为 $a(b(f(a,4),b(3,f)),f(2,6))$, 其中各个结点按照出现顺序编号为 1, 2, 3, ..., 即编号 k 表示目前为止写下的第 k 个结点。

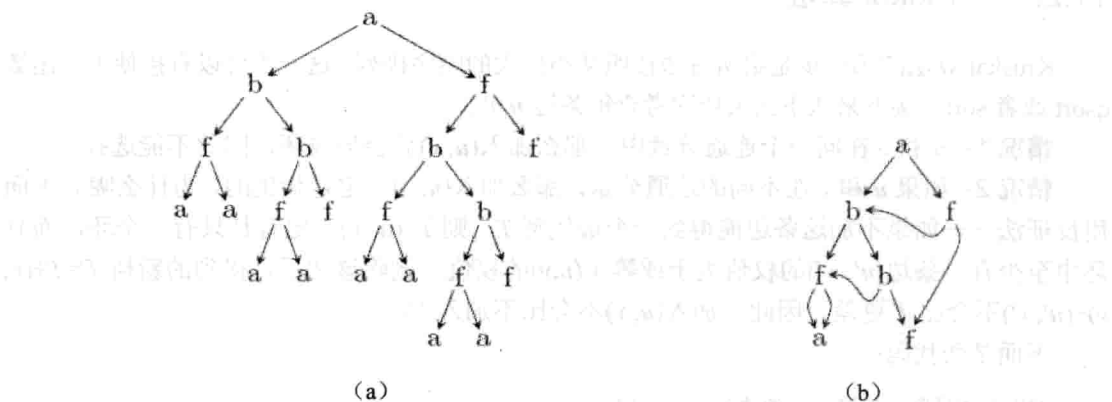


图 11-3 公共表达式消除

输入一个长度不超过 50000 的表达式, 输出一个等价的, 结点最少的图。

【分析】

算法的第一步是构造表达式树。接下来应该怎么做呢? 是否可以用两两比较的方法去掉重复? 比较两棵树的时间复杂度为 $O(n)$ (因为要递归比较二者的所有后代), 再加上二重循环枚举两棵子树, 总时间复杂度高达 $O(n^3)$, 无法承受。此处不仅需要更快地比较两棵树, 还需要更快地查找一棵树是否存在过。

借用第 5 章“集合栈计算机”的思路, 用一个 map 把子树映射成编号 1, 2, ...。这样一来, 子树就可以用根的名字 (字符串) 和左右子结点编号表示。如图 11-4 所示, 用 $(a,0,0)$ 表示根的名字为 a , 且左右子结点均为空 (0 表示不存在) 的子树, 即叶子 a 。可以看到, 下面所有叶子 a 的编号都是 4。再例如, $(b,3,6)$ 就是根的名字为 b , 左右两个子树的编号分别为 3, 6。可以看到, 这样的子树编号均为 5。

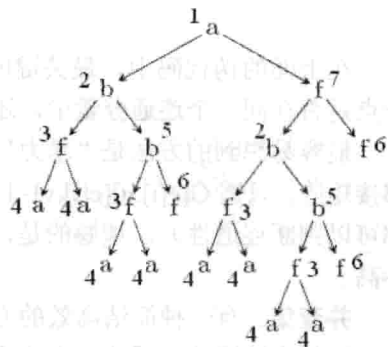


图 11-4 子树编号

这样, 每次判断一棵子树是否出现过只需要在 map 中查找, 总时间复杂度为 $O(n \log n)$ 。

11.2 最小生成树

前面提到过, 在无向图中, 连通且不含圈的图称为树 (Tree)。给定无向图 $G=(V,E)$,

连接 G 中所有点，且边集是 E 的子集的树称为 G 的生成树（Spanning Tree），而权值最小的生成树称为最小生成树（Minimal Spanning Tree, MST）。构造 MST 的算法有很多，最常见的有两个：Kruskal 算法和 Prim 算法。限于篇幅，这里只介绍 Kruskal 算法，它易于编写，而且效率很高。

11.2.1 Kruskal 算法

Kruskal 算法的第一步是给所有边按照从小到大的顺序排列。这一步可以直接使用库函数 `qsort` 或者 `sort`。接下来从小到大依次考查每条边 (u, v) 。

情况 1: u 和 v 在同一个连通分量中，那么加入 (u, v) 后会形成环，因此不能选择。

情况 2: 如果 u 和 v 在不同的连通分量，那么加入 (u, v) 一定是最优的。为什么呢？下面用反证法——如果不加这条边能得到一个最优解 T ，则 $T+(u, v)$ 一定有且只有一个环，而且环中至少有一条边 (u', v') 的权值大于或等于 (u, v) 的权值。删除该边后，得到的新树 $T'=T+(u, v)-(u', v')$ 不会比 T 更差。因此，加入 (u, v) 不会比不加入差。

下面是伪代码：

把所有边排序，记第 i 小的边为 $e[i]$ ($1 \leq i \leq m$)

初始化 MST 为空

初始化连通分量，让每个点自成一个独立的连通分量

```
for(int i = 0; i < m; i++)
```

```
    if( $e[i].u$  和  $e[i].v$  不在同一个连通分量) {
```

```
        把边  $e[i]$  加入 MST
```

```
        合并  $e[i].u$  和  $e[i].v$  所在的连通分量
```

```
    }
```

在上面的伪代码中，最关键的地方在于“连通分量的查询与合并”：需要知道任意两个点是否在同一个连通分量中，还需要合并两个连通分量。

最容易想到的方法是“暴力”——每次“合并”时只在 MST 中加入一条边（如果使用邻接矩阵，只需 $G[e[i].u][e[i].v]=1$ ），而“查询”时直接在 MST 中进行图遍历（DFS 和 BFS 都可以判断连通性）。遗憾的是，这个方法不仅复杂（需要写 DFS 或者 BFS），而且效率不高。

并查集。有一种简洁高效的方法可用来处理这个问题：使用并查集（Union-Find Set）。可以把每个连通分量看成一个集合，该集合包含了连通分量中的所有点。这些点两两连通，而具体的连通方式无关紧要，就好比集合中的元素没有先后顺序之分，只有“属于”和“不属于”的区别。在图中，每个点恰好属于一个连通分量，对应到集合表示中，每个元素恰好属于一个集合。换句话说，图的所有连通分量可以用若干个不相交集合来表示。

并查集的微妙之处在于用树来表示集合。例如，若包含点 1, 2, 3, 4, 5, 6 的图有 3 个连通分量 {1,3}、{2,5,6}、{4}，则需要用 3 棵树来表示。这 3 棵树的具体形态无关紧要，只要有一棵树包含 1、3 两个点，一棵树包含 2、5、6 这 3 个点，还有一棵树只包含 4 这一点即可。规定每棵树的根结点是这棵树所对应的集合的代表元（representative）。

如果把 x 的父结点保存在 $p[x]$ 中（如果 x 没有父结点，则 $p[x]$ 等于 x ），则不难写出“查找结点 x 所在树的根结点”的递归程序：`int find(int x) { p[x] = x ? x : find(p[x]); }`，通俗地讲就是：如果 $p[x]$ 等于 x ，说明 x 本身就是树根，因此返回 x ；否则返回 x 的父结点 $p[x]$ 所在树的树根。

问题来了：在特殊情况下，这棵树可能是一条长长的链。设链的最后一个结点为 x ，则每次执行 `find(x)` 都会遍历整条链，效率十分低下。看上去是个很棘手的问题，其实改进方法很简单。既然每棵树表示的只是一个集合，因此树的形态是无关紧要的，并不需要在“查找”操作之后保持树的形态不变，只要顺便把遍历过的结点都改成树根的子结点，下次查找就会快很多了，如图 11-5 所示。

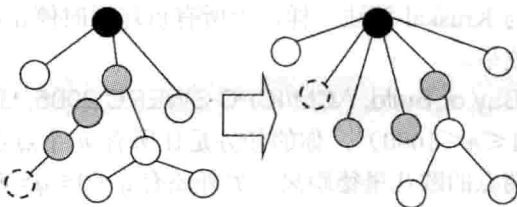


图 11-5 并查集中的路径压缩

这样，Kruskal 算法的完整代码便不难给出了。假设第 i 条边的两个端点序号和权值分别保存在 $u[i]$, $v[i]$ 和 $w[i]$ 中，而排序后第 i 小的边的序号保存在 $r[i]$ 中（这叫做间接排序。排序的关键字是对象的“代号”，而不是对象本身）。

```
int cmp(const int i, const int j) { return w[i] < w[j]; } //间接排序函数
int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } //并查集的 find
int Kruskal() {
    int ans = 0;
    for(int i = 0; i < n; i++) p[i] = i; //初始化并查集
    for(int i = 0; i < m; i++) r[i] = i; //初始化边序号
    sort(r, r+m, cmp); //给边排序
    for(int i = 0; i < m; i++) {
        int e = r[i]; int x = find(u[e]); int y = find(v[e]);
        //找出当前边两个端点所在集合编号
        if(x != y) { ans += w[e]; p[x] = y; } //如果在不同集合，合并
    }
    return ans;
}
```

注意， x 和 y 分别是第 e 条边的两个端点所在连通分量的代表元。合并 x 和 y 所在集合可以简单地写成 $p[x]=y$ ，即直接把 x 作为 y 的子结点，则两个树就合并成一棵树了。注意不能写成 $p[u[e]]=p[v[e]]$ ，因为 $u[e]$ 和 $v[e]$ 不一定是树根。并查集的效率非常高，在平摊意义下，`find` 函数的时间复杂度几乎可以看成是常数（而 `union` 显然是常数时间）。

11.2.2 竞赛题目选解

例题 11-2 苗条的生成树 (Slim Span, ACM/ICPC Japan 2007, UVa1395)

给出一个 n ($n \leq 100$) 结点的图，求苗条度（最大边减最小边的值）尽量小的生成树。

【分析】

首先把边按权值从小到大排序。对于一个连续的边集区间 $[L, R]$ ，如果这些边使得 n 个点全部连通，则一定存在一个苗条度不超过 $W[R] - W[L]$ 的生成树（其中 $W[i]$ 表示排序后第 i 条边的权值）。

从小到大枚举 L ，对于每个 L ，从小到大枚举 R ，同时用并查集将新进入 $[L, R]$ 的边两端的点合并成一个集合，与 Kruskal 算法一样。当所有点连通时停止枚举 R ，换下一个 L （并且把 R 重置为 L ）继续枚举。

例题 11-3 买还是建 (Buy or Build, ACM/ICPC SWERC 2005, UVa1151)

平面上有 n 个点 ($1 \leq n \leq 1000$)，你的任务是让所有 n 个点连通。为此，你可以新建一些边，费用等于两个端点的欧几里德距离。另外还有 q ($0 \leq q \leq 8$) 个“套餐”可以购买，如果你购买了第 i 个套餐，该套餐中的所有结点将变得相互连通。第 i 个套餐的花费为 C_i 。如图 11-6 所示，一共有 3 个套餐：

它的最优解是购买套餐 1 和套餐 2，然后手动连接两条边，如图 11-7 所示。

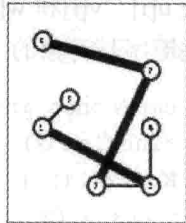
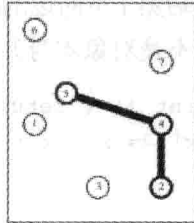
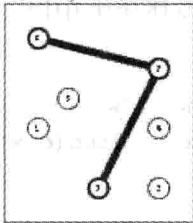
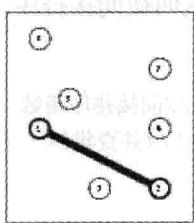


图 11-6 3 个套餐

图 11-7 购买套餐 1、2 并连接边

【分析】

最容易想到的算法是：先枚举购买哪些套餐，把套餐中包含的边的权值设为 0，然后求最小生成树。由于枚举量为 $O(2^q)$ ，给边排序的时间复杂度为 $O(n^2 \log n)$ ，而排序之后每次 Kruskal 算法的时间复杂度为 $O(n^2)$ ，因此总时间复杂度为 $O(2^q n^2 + n^2 \log n)$ ，对于题目的规模来说太大了。

只需一个小小的优化即可降低时间复杂度：先求一次原图（不购买任何套餐）的最小生成树，得到 $n-1$ 条边，然后每次枚举完套餐后只考虑套餐中的边和这 $n-1$ 条边，则枚举套餐之后再求最小生成树时，图上的边已经寥寥无几。

为什么可以这样呢？首先回顾一下，在 Kruskal 算法中，哪些边不会进入最小生成树。答案是：两端已经属于同一个连通分量的边。买了套餐以后，相当于一些边的权变为 0，而对于不在套餐中的每条边 e ，排序在 e 之前的边一个都没少，反而可能多了一些权值为 0 的边，所以在原图 Kruskal 时被“扔掉”的边，在后面的 Kruskal 中也一样会被扔掉。

本题还有一个地方需要说明：因为 Kruskal 在连通分量包含 n 个点时会终止，所以对于

随机数据, 即使用原始的“暴力算法”, 也能很快出解。如果你是命题者, 可以这样出一个数据: 有一个点很远, 而其他 $n-1$ 个点相互比较近。这样, 相距较近的 $n-1$ 个点之间的 $C(n-1, 2)$ 条边会排序在前面, 每次 Kruskal 都会先考虑完所有这些边。而考虑这些边时是无法让远点和近点连通的。

11.3 最短路问题

最短路问题并不陌生: 在第 9 章中, 曾介绍过无权和带权 DAG 上的最短路和最长路, 二者的算法几乎是一样的 (只是初始化不同, 并且状态转移时把 min 和 max 互换)。但如果图中可以有环, 情况就不同了。

11.3.1 Dijkstra 算法

Dijkstra 算法适用于边权为正的情况。下面直接给出 Dijkstra 算法的伪代码, 它可用于计算正权图上的单源最短路 (Single-Source Shortest Paths, SSSP), 即从单个源点出发, 到所有结点的最短路。该算法同时适用于有向图和无向图。

清除所有点的标号

设 $d[0]=0$, 其他 $d[i]=\text{INF}$

循环 n 次 {

 在所有未标号结点中, 选出 d 值最小的结点 x

 给结点 x 标记

 对于从 x 出发的所有边 (x, y) , 更新 $d[y] = \min\{d[y], d[x]+w(x, y)\}$

}

下面是伪代码对应的程序。假设起点是结点 0, 它到结点 i 的路径长度为 $d[i]$ 。未标号结点的 $v[i]=0$, 已标号结点的 $v[i]=1$ 。为了简单起见, 用 $w[x][y]=\text{INF}$ 表示边 (x, y) 不存在。

```
memset(v, 0, sizeof(v));
for(int i = 0; i < n; i++) d[i] = (i==0 ? 0 : INF);
for(int i = 0; i < n; i++) {
    int x, m = INF;
    for(int y = 0; y < n; y++) if(!v[y] && d[y]<=m) m = d[x=y];
    v[x] = 1;
    for(int y = 0; y < n; y++) d[y] = min(d[y], d[x] + w[x][y]);
}
```

除了求出最短路的长度外, 使用 Dijkstra 算法也能很方便地打印出结点 0 到所有结点的最短路本身, 原理和动态规划中的方案打印一样——从终点出发, 不断顺着 $d[i]+w[i][j]=d[j]$ 的边 (i, j) 从结点 j “退回”到结点 i , 直到回到起点。另外, 仍然可以用空间换时间, 在更新 d 数组时维护“父亲指针”。具体来说, 需要把 $d[y] = \min(d[y], d[x]+w[x][y])$ 改成:

```

if(d[y] > d[x] + w[x][y]) {
    d[y] = d[x] + w[x][y];
    fa[y] = x;
}

```

这称为边 (x,y) 上的松弛操作（relaxation）。不难看出，上面程序的时间复杂度为 $O(n^2)$ ——循环体一共执行了 n 次，而在每次循环中，“求最小 d 值”和“更新其他 d 值”均是 $O(n)$ 的。由于最短路算法实在太重要了，下面花一些篇幅把它优化到 $O(m\log n)$ ，并给出一份简单高效的完整代码。

等一等，为什么说是“优化到”呢？在最坏情况下， m 和 n^2 是同阶的， $m\log n$ 岂不是比 n^2 要大？这话没错，但在很多情况下，图中的边并没有那么多， $m\log n$ 比 n^2 小得多。 m 远小于 n^2 的图称为稀疏图（Sparse Graph），而 m 相对较大的图称为稠密图（Dense Graph）。

和前面一样，稀疏图适合使用 vector 数组保存。除此之外，还有一种流行的表示法——邻接表（Adjacency List）。在这种表示法中，每个结点 i 都有一个链表，里面保存着从 i 出发的所有边。对于无向图来说，每条边会在邻接表中出现两次。和前面一样，这里继续用数组实现链表：首先给每条边编号，然后用 $\text{first}[u]$ 保存结点 u 的第一条边的编号， $\text{next}[e]$ 表示编号为 e 的边的“下一条边”的编号。下面的函数读入有向图的边列表，并建立邻接表：

```

int n, m;
int first[maxn];
int u[maxm], v[maxm], w[maxm], next[maxm];
void read_graph() {
    scanf("%d%d", &n, &m);
    for(int i = 0; i < n; i++) first[i] = -1; //初始化表头
    for(int e = 0; e < m; e++) {
        scanf("%d%d%d", &u[e], &v[e], &w[e]);
        next[e] = first[u[e]]; //插入链表
        first[u[e]] = e;
    }
}

```

上述代码的巧妙之处是插入到链表的首部而非尾部，这样就避免了对链表的遍历。不过需要注意的是，同一个起点的各条边在邻接表中的顺序和读入顺序正好相反。读者如果还记得哈希表，应该会发现这里的链表和哈希表中的链表实现很相似。

尽管邻接表很流行，但在概念上 vector 数组更为简单，所以接下来仍然给出基于 vector 数组的代码。虽然在最短路问题中，每条边只有“边权”这一个属性，但后面的最大流以及最小费用流中还会出现“容量”、“流量”以及“费用”等属性。所以在这里使用一个称为 Edge 的结构体，这会让这里的代码与后面的代码在风格上更统一。

```

struct Edge {
    int from, to, dist;
    Edge(int u, int v, int d):from(u),to(v),dist(d) {}
};

```

为了使用方便, 此处把算法中用到的数据结构封装到一个结构体中:

```
struct Dijkstra {
    int n, m;
    vector<Edge> edges;
    vector<int> G[maxn];
    bool done[maxn];    //是否已永久标号
    int d[maxn];        //s 到各个点的距离
    int p[maxn];        //最短路中的上一条弧

    void init(int n) {
        this->n = n;
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int dist) {
        edges.push_back(Edge(from, to, dist));
        m = edges.size();
        G[from].push_back(m-1);
    }

    void dijkstra(int s) {
        ...
    }
};
```

不难看出, 在 `vector` 数组中保存的只是边的编号。有了编号之后可以从 `edges` 数组中查到边的具体信息。有了这样的数据结构, “遍历从 x 出发的所有边 (x,y) , 更新 $d[y]$ ” 就可以写成 “`for(int i = 0; i < G[u].size(); i++)` 执行边 `edges[G[u][i]]` 上的松弛操作”。尽管在最坏情况下, 这个循环仍然会循环 $n-1$ 次, 但从整体上来看, 每条边恰好被检查过一次 (想一想, 为什么), 因此松弛操作执行的次数恰好是 m 。这样, 只需集中精力优化 “找出未标号结点中的最小 d 值” 即可。

在 Dijkstra 算法中, $d[i]$ 越小, 应该越先出队, 因此需要使用自定义比较器。在 STL 中, 可以用 `greater<int>` 表示 “大于” 运算符, 因此可以用 `priority_queue<int, vector<int>, greater<int>>` 来声明一个小整数先出队的优先队列。然而, 除了需要最小的 d 值之外, 还要找到这个最小值对应的结点编号, 所以要把 d 值和编号 “捆绑” 成一个整体放到优先队列中, 使得取出最小 d 值的同时也会取出对应的结点编号。

STL 中的 `pair` 便是专门把两个类型捆绑到一起的。为了方便起见, 用 `typedef pair<int,int> pii` 自定义一个 `pii` 类型, 则 `priority_queue<pii, vector<pii>, greater<pii>>` 就定义了一个由二元组构成的优先队列。`pair` 定义了它自己的排序规则——先比较第一维, 相等时才比较第二维, 因此需要按 $(d[i], i)$ 而不是 $(i, d[i])$ 的方式组合。这样的方法理论上和实际上都没有问题,

很多用户并不习惯。为了保持简单，这里不使用 `pair`，而是显式定义一个结构体作为优先队列中的元素类型，例如：

```
struct HeapNode {
    int d, u;
    bool operator < (const HeapNode& rhs) const {
        return d > rhs.d;
    }
};
```

然后主算法就可以写出来了：

```
void dijkstra(int s) {
    priority_queue<HeapNode> Q;
    for(int i = 0; i < n; i++) d[i] = INF;
    d[s] = 0;
    memset(done, 0, sizeof(done));
    Q.push((HeapNode){0, s});
    while(!Q.empty()) {
        HeapNode x = Q.top(); Q.pop();
        int u = x.u;
        if(done[u]) continue;
        done[u] = true;
        for(int i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if(d[e.to] > d[u] + e.dist) {
                d[e.to] = d[u] + e.dist;
                p[e.to] = G[u][i];
                Q.push((HeapNode){d[e.to], e.to});
            }
        }
    }
}
```

在松弛成功后，需要修改结点 `e.to` 的优先级，但 STL 中的优先队列不提供“修改优先级”的操作。因此，只能将新元素重新插入优先队列。这样做并不会影响结果的正确性，因为 `d` 值小的结点自然会先出队。为了防止结点的重复扩展，如果发现新取出来的结点曾经被取出来过（`done[u]`），应该直接把它扔掉。避免重复的另一个方法是把 `if(done[u])` 改成 `if(x.d != d[u])`，可以省掉一个 `done` 数组。

再补充一点：即使是稠密图，使用 `priority_queue` 实现的 Dijkstra 算法也常常比基于邻接矩阵的 Dijkstra 算法的运算速度快。理由很简单，执行 `push` 操作的前提是 `d[e.to] > d[u] + e.dist`，如果这个式子常常不成立，则 `push` 操作会很少。

11.3.2 Bellman-Ford 算法

当负权存在时, 连最短路都不一定存在了。尽管如此, 还是有办法在最短路存在的情况下把它求出来。在介绍算法之前, 请读者确认这样一个事实: 如果最短路存在, 一定存在一个不含环的最短路。

理由如下: 在边权可正可负的图中, 环有零环、正环和负环 3 种。如果包含零环或正环, 去掉以后路径不会变长; 如果包含负环, 则意味着最短路不存在(想一想, 为什么)。

既然不含环, 最短路最多只经过(起点不算) $n-1$ 个结点, 可以通过 $n-1$ “轮” 松弛操作得到, 像这样(起点仍然是 0):

```
for(int i = 0; i < n; i++) d[i] = INF;
d[0] = 0;
for(int k = 0; k < n-1; k++) //迭代 n-1 次
    for(int i = 0; i < m; i++) //检查每条边
    {
        int x = u[i], y = v[i];
        if(d[x] < INF) d[y] = min(d[y], d[x]+w[i]); //松弛
    }
```

上述算法称为 Bellman-Ford 算法, 不难看出它的时间复杂度为 $O(nm)$ 。在实践中, 常常用 FIFO 队列来代替上面的循环检查, 像这样:

```
bool bellman_ford(int s) {
    queue<int> Q;
    memset(inq, 0, sizeof(inq));
    memset(cnt, 0, sizeof(cnt));
    for(int i = 0; i < n; i++) d[i] = INF;
    d[s] = 0;
    inq[s] = true;
    Q.push(s);

    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        inq[u] = false;
        for(int i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if(d[u] < INF && d[e.to] > d[u] + e.dist) {
                d[e.to] = d[u] + e.dist;
                p[e.to] = G[u][i];
                if(!inq[e.to]) { Q.push(e.to); inq[e.to] = true; if(++cnt[e.to] > n) return false; }
            }
        }
    }
```

```

    }
  }
  return true;
}

```

有没有注意到上面的代码和前面的 Dijkstra 算法很像？一方面，优先队列替换为了普通的 FIFO 队列，而另一方面，一个结点可以多次进入队列。可以证明，采取 FIFO 队列的 Bellman-Ford 算法在最坏情况下需要 $O(nm)$ 时间，不过在实践中，往往只需要很短的时间就能求出最短路。上面的代码还有一个功能：在发现负圈时及时退出。注意，这只说明 s 可以到达一个负圈，并不代表 s 到每个点的最短路都不存在。另外，如果图中有其他负圈但是 s 无法到达这个负圈，则上面的算法也无法找到。解决方法留给读者思考（提示：加一个结点）。

11.3.3 Floyd 算法

如果要求出每两点之间的最短路，不必调用 n 次 Dijkstra（边权均为正）或者 Bellman-ford（有负权）。有一个更简单的方法可以实现——Floyd-Warshall 算法（请记住下面的代码！）：

```

for(int k = 0; k < n; k++)
  for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
      d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

```

在调用它之前只需做一些简单的初始化： $d[i][i]=0$ ，其他 d 值为“正无穷”INF。注意这里有一个潜在的问题：如果 INF 定义太大（如 2000000000），加法 $d[i][k]+d[k][j]$ 可能会溢出！但如果 INF 太小，可能会使得长度为 INF 的边真的变成最短路的一部分。谨慎起见，最好估计一下实际最短路长度的上限，并把 INF 设置成“只比它大一点点”的值。例如，最多有 1000 条边，若每条边长度不超过 1000，可以把 INF 设成 1000001。

如果坚持认为不应该允许 INF 和其他值相加，更不应该得到一个大于 INF 的数，请把上述代码改成：

```

for(int k = 0; k < n; k++)
  for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
      if(d[i][j] < INF && d[k][j] < INF)
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

```

在有向图中，有时不必关心路径的长度，而只关心每两点间是否有通路，则可以用 1 和 0 分别表示“连通”和“不连通”。这样，除了预处理需做少许调整外，主算法中只需把“ $d[i][j] = \min\{d[i][j], d[i][k] + d[k][j]\}$ ”改成“ $d[i][j] = d[i][j] \vee (d[i][k] \wedge d[k][j])$ ”。这样的结果称为有向图的传递闭包（Transitive Closure）。

11.3.4 竞赛题目选讲

例题 11-4 电话圈 (Calling Circles, ACM/ICPC World Finals 1996, UVa247)

如果两个人相互打电话 (直接或间接), 则说他们在同一个电话圈里。例如, a 打给 b, b 打给 c, c 打给 d, d 打给 a, 则这 4 个人在同一个圈里; 如果 e 打给 f 但 f 不打给 e, 则不能推出 e 和 f 在同一个电话圈里。输入 n ($n \leq 25$) 个人的 m 次电话, 找出所有电话圈。人名只包含字母, 不超过 25 个字符, 且不重复。

【分析】

首先用 floyd 求出传递闭包, 即 $g[i][j]$ 表示 i 是否直接或者间接给 j 打过电话, 则当且仅当 $g[i][j]=g[j][i]=1$ 时二者处于一个电话圈。构造一个新图, 在“在一个电话圈里”的两个人之间连一条边, 然后依次输出各个连通分量的所有人即可。

例题 11-5 噪音恐惧症 (Audiophobia, UVa10048)

输入一个 C 个点 S 条边 ($C \leq 100, S \leq 1000$) 的无向带权图, 边权表示该路径上的噪声值。当噪声值太大时, 耳膜可能会受到伤害, 所以当你从某点去往另一个点时, 总是希望路上经过的最大噪声值最小。输入一些询问, 每次询问两个点, 输出这两点间最大噪声值最小的路径。例如, 在图 11-8 中, A 到 G 的最大噪声值为 80, 是所有其他路径中最小的 (如 ABEG 的最大噪声值为 90)。

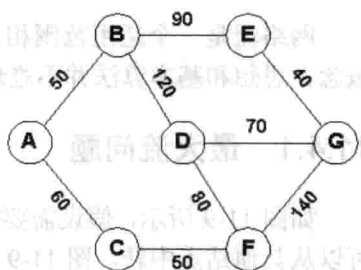


图 11-8 路径与噪声值

【分析】

本题的做法十分简单: 直接用 floyd 算法, 但是要把加法改成 min, min 改成 max。为什么可以这样做呢? 不管是 floyd 算法还是 dijkstra 算法, 都是基于这样一个事实: 对于任意一条至少包含两条边的路径 $i \rightarrow j$, 一定存在一个中间点 k , 使得 $i \rightarrow j$ 的总长度等于 $i \rightarrow k$ 与 $k \rightarrow j$ 的长度之和。对于不同的点 k , $i \rightarrow k$ 和 $k \rightarrow j$ 的长度之和可能不同, 最后还需要取一个最小值才是 $i \rightarrow j$ 的最短路径。把刚才的推理中“之和”与“取最小值”换成“取最小值”和“取最大值”, 推理仍然适用。

例题 11-6 这不是 bug, 而是特性 (It's not a Bug, it's a Feature!, UVa 658)

补丁在修正 bug 时, 有时也会引入新的 bug。假定有 n ($n \leq 20$) 个潜在 bug 和 m ($m \leq 100$) 个补丁, 每个补丁用两个长度为 n 的字符串表示, 其中字符串的每个位置表示一个 bug。第一个串表示打补丁之前的状态 (“-”表示该 bug 必须不存在, “+”表示必须存在, 0 表示无所谓), 第二个串表示打补丁之后的状态 (“-”表示不存在, “+”表示存在, 0 表示不变)。每个补丁都有一个执行时间, 你的任务是用最少的时间把一个所有 bug 都存在的软件通过打补丁的方式变得没有 bug。一个补丁可以打多次。

【分析】

在任意时刻, 每个 bug 可能存在也可能不存在, 所以可以用一个 n 位二进制串表示当前软件的“状态”。打完补丁之后, bug 状态会发生改变, 对应“状态转移”。是不是很像动态规划? 可惜动态规划是行不通的, 因为状态经过多次转移之后可能会回到以前的状态,

即状态图并不是 DAG。如果直接用记忆化搜索，会出现无限递归。

正确的方法是把状态看成结点，状态转移看成边，转化成图论中的最短路径问题，然后使用 Dijkstra 或 Bellman-Ford 算法求解。不过这题和普通的最短路径问题不一样：结点很多，多达 2^n 个，而且很多状态根本遇不到（即不管怎么打补丁，也不可能打成那个状态），所以没有必要像前面那样先把图储存好。

还记得第 7 章中介绍的“隐式图搜索”吗？这里也可以用相同的方法：当需要得到某个结点 u 出发的所有边时，不是去读 $G[u]$ ，而是直接枚举所有 m 个补丁，看看是否能打得上。不管是 Dijkstra 算法还是 Bellman-Ford 算法，这个方法都适用。本题很经典，强烈建议读者编程实现。

11.4 网络流初步

网络流是一个适用范围相当广的模型，相关的算法也非常多。尽管如此，网络流中的概念、思想和基本算法并不难理解。

11.4.1 最大流问题

如图 11-9 所示，假设需要把一些物品从结点 s （称为源点）运送到结点 t （称为汇点），可以从其他结点中转。图 11-9 (a) 中各条有向边的权表示最多能有多少个物品从这条边的起点直接运送到终点。例如，最多可以有 9 个物品从结点 v_3 运送到 v_2 。

图 11-9 (b) 展示了一种可能的方案，其中每条边中的第一个数字表示实际运送的物品数目，而第二个数字就是题目中的上限。

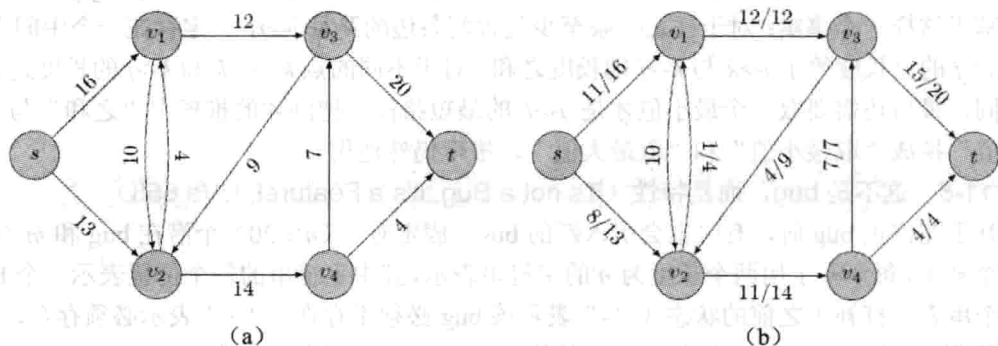


图 11-9 物资运送问题

这样的问题称为最大流问题（Maximum-Flow Problem）。对于一条边 (u,v) ，它的物品上限称为容量（capacity），记为 $c(u,v)$ （对于不存在的边 (u,v) ， $c(u,v)=0$ ）；实际运送的物品称为流量（flow），记为 $f(u,v)$ 。注意，“把 3 个物品从 u 运送到 v ，又把 5 个物品从 v 运送到 u ”没什么意义，因为它等价于把两个物品从 v 运送到 u 。这样，就可以规定 $f(u,v)$ 和 $f(v,u)$ 最多只有一个正数（可以均为 0），并且 $f(u,v)=-f(v,u)$ 。这样规定就好比“把 3 个物

品从 u 运送到 v ”等价于“把 -3 个物品从 v 运送到 u ”一样。

最大流问题的目标是把最多的物品从 s 运送到 t ，而其他结点都只是中转，因此对于除了结点 s 和 t 外的任意结点 u ， $\sum_{(u,v) \in E} f(u,v) = 0$ （这些 f 中有些是负数）。从 s 运送出来的物品数目等于到达 t 的物品数目，而这正是此处最大化的目标。

提示 11-2：在最大流问题中，容量 c 和流量 f 满足 3 个性质：容量限制 ($f(u,v) \leq c(u,v)$)、斜对称性 ($f(u,v) = -f(v,u)$) 和流量平衡 (对于除了结点 s 和 t 外的任意结点 u ， $\sum_{(u,v) \in E} f(u,v) = 0$)。

问题的目标是最大化 $|f| = \sum_{(s,v) \in E} f(s,v) = \sum_{(u,t) \in E} f(u,t)$ ，即从 s 点流出的净流量（它也等于流入 t 点的净流量）。

11.4.2 增广路算法

介绍完最大流问题后，下面介绍求解最大流问题的算法。算法思想很简单，从零流（所有边的流量均为 0）开始不断增加流量，保持每次增加流量后都满足容量限制、斜对称性和流量平衡 3 个条件。

计算出图 11-10 (a) 中的每条边上容量与流量之差（称为残余容量，简称残量），得到图 11-10 (b) 中的残量网络 (residual network)。同理，由图 11-10 (c) 可以得到图 11-10 (d)。注意残量网络中的边数可能达到原图中边数的两倍，如原图中 $c=16, f=11$ 的边在残量网络中对应正反两条边，残量分别为 $16-11=5$ 和 $0-(-11)=11$ 。

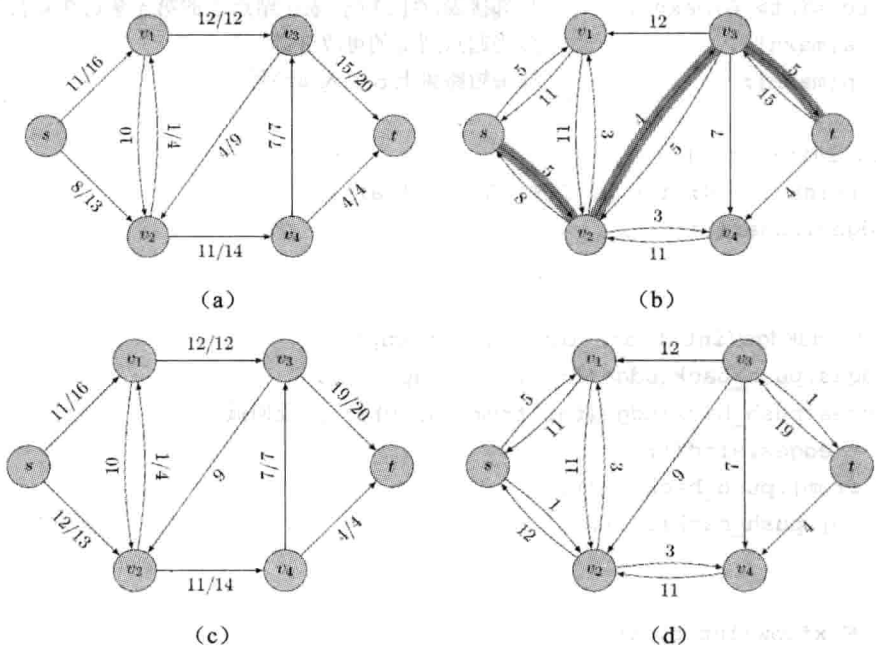


图 11-10 残量网络和增广路算法

该算法基于这样一个事实：残量网络中任何一条从 s 到 t 的有向道路都对应一条原图中

的增广路 (augmenting path) ——只要求出该道路中所有残量的最小值 d , 把对应的所有边上的流量增加 d 即可, 这个过程称为增广 (augmenting)。不难验证, 如果增广前的流量满足 3 个条件, 增广后仍然满足。显然, 只要残量网络中存在增广路, 流量就可以增大。可以证明它的逆命题也成立: 如果残量网络中不存在增广路, 则当前流就是最大流。这就是著名的增广路定理。

提示 11-3: 当且仅当残量网络中不存在 s - t 有向道路 (增广路) 时, 此时的流是从 s 到 t 的最大流。

“找任意路径”最简单的办法无疑是用 DFS, 但很容易找出让它很慢的例子。一个稍微好一些的方法是使用 BFS, 它足以应对数据不刁钻的网络流题目。这就是 Edmonds-Karp 算法。下面是完整的代码。注意 Edge 结构体多了 flow 和 cap 两个变量, 但是 AddEdge 却和 Dijkstra 中的同名函数很接近。这便是得益于 Edge 结构体这一设计。

```
struct Edge {
    int from, to, cap, flow;
    Edge(int u, int v, int c, int f):from(u),to(v),cap(c),flow(f) {}
};

struct EdmondsKarp {
    int n, m;
    vector<Edge> edges;           //边数的两倍
    vector<int> G[maxn];         //邻接表, G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    int a[maxn];                 //当起点到 i 的可改进量
    int p[maxn];                 //最短路树上 p 的入弧编号

    void init(int n) {
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap) {
        edges.push_back(Edge(from, to, cap, 0));
        edges.push_back(Edge(to, from, 0, 0)); //反向弧
        m = edges.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }

    int Maxflow(int s, int t) {
        int flow = 0;
        for(;;) {
            memset(a, 0, sizeof(a));
```

```

queue<int> Q;
Q.push(s);
a[s] = INF;
while(!Q.empty()) {
    int x = Q.front(); Q.pop();
    for(int i = 0; i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if(!a[e.to] && e.cap > e.flow) {
            p[e.to] = G[x][i];
            a[e.to] = min(a[x], e.cap - e.flow);
            Q.push(e.to);
        }
    }
    if(a[t]) break;
}
if(!a[t]) break;
for(int u = t; u != s; u = edges[p[u]].from) {
    edges[p[u]].flow += a[t];
    edges[p[u]^1].flow -= a[t];
}
flow += a[t];
}
return flow;
}
};

```

注意上面代码中的一个技巧：每条弧和对应的反向弧保存在一起。边 0 和 1 互为反向边；边 2 和 3 互为反向边……一般地，边 i 的反向边为 i^1 ，其中“ $\wedge$ ”为二进制异或运算符（想一想，为什么）。

正如所见，上面的代码和普通 BFS 并没有太大的不同。唯一需要注意的是，在扩展结点的同时还需递推出从 s 到每个结点 i 的路径上的最小残量 $a[i]$ ，则 $a[t]$ 就是整条 $s-t$ 道路上的最小残量。另外，由于 $a[i]$ 总是正数，所以用它代替了原来的 vis 标志数组。上面的代码把流初始化为零流，但这并不是必需的。只要初始流是可行的（满足 3 个限制条件），就可以用增广路算法进行增广。

11.4.3 最小割最大流定理

有一个与最大流关系密切的问题：最小割。如图 11-11 所示，把所有顶点分成两个集合 S 和 $T=V-S$ ，其中源点 s 在集合 S 中，汇点 t 在集合 T 中。

如果把“起点在 S 中，终点在 T 中”的边全部删除，就无法从 s 到达 t 了。这样的集合划分 (S, T) 称为一个 $s-t$ 割，它的容量定义为： $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$ ，即起点在 S 中，终点在 T 中的所有边的容量和。

还可从另外一个角度看待割。如图 11-12 所示, 从 s 运送到 t 的物品必然通过跨越 S 和 T 的边, 所以从 s 到 t 的净流量等于 $|f| = f(S, T) = \sum_{u \in S, v \in T} f(u, v) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$ 。

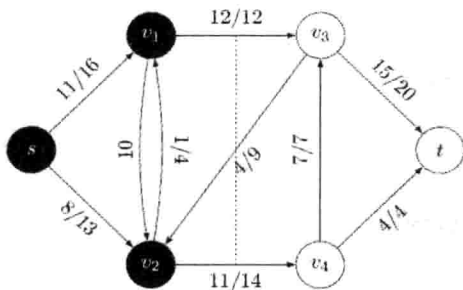


图 11-11 网络中的割

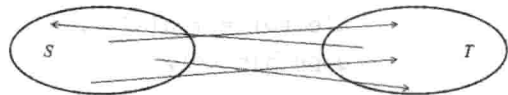


图 11-12 流和割的关系

注意这里的割 (S, T) 是任取的, 因此得到了一个重要结论: 对于任意 $s-t$ 流 f 和任意 $s-t$ 割 (S, T) , 有 $|f| \leq c(S, T)$ 。

下面来看残量网络中没有增广路的情形。既然不存在增广路, 在残量网络中 s 和 t 并不连通。当 BFS 没有找到任何 $s-t$ 道路时, 把已标号结点 ($a[u] > 0$ 的结点 u) 集合看成 S , 令 $T = V - S$, 则在残量网络中 S 和 T 分离, 因此在原图中跨越 S 和 T 的所有弧均满载 (这样的边才不会存在于残量网络中), 且没有从 T 回到 S 的流量, 因此 $|f| \leq c(S, T)$ 成立。

前面说过, 对于任意的 f 和 (S, T) , 都有 $|f| \leq c(S, T)$, 而此处又找到了一组让等号成立的 f 和 (S, T) 。这样, 便同时证明了增广路定理和最小割最大流定理: 在增广路算法结束时, f 是 $s-t$ 最大流, (S, T) 是 $s-t$ 最小割。

提示 11-4: 增广路算法结束时, 令已标号结点 ($a[u] > 0$ 的结点) 集合为 S , 其他结点集合为 $T = V - S$, 则 (S, T) 是图的 $s-t$ 最小割。

11.4.4 最小费用最大流问题

下面给网络流增加一个因素: 费用。假设每条边除了有一个容量限制外, 还有一个单位流量所需的费用 (cost)。图 11-13 (a) 中分别用 c 和 a 来表示每条边的容量和费用, 而图 11-13 (b) 给出了一个在总流量最大的前提下, 总费用最小的流 (费用为 10), 即最小费用最大流。另一个最大流是从 s 分别运送一个单位到 x 和 y , 但总费用为 11, 不是最优。

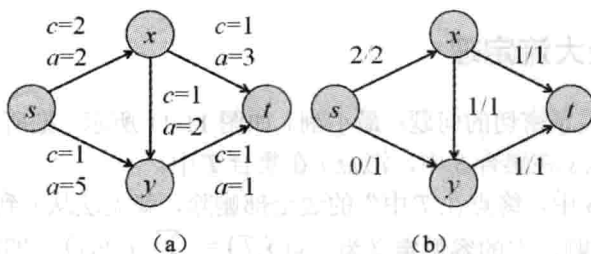


图 11-13 最小费用最大流

在最小费用流问题中, 平行边变得有意义了: 可能会有两条从 u 到 v 的弧, 费用分别为 1 和 2。在没有费用的情况下, 可以把二者合并, 但由于费用的出现, 无法合并这两条弧。再如, 若边 (u, v) 和 (v, u) 均存在, 且费用都是负数, 则“同时从 u 流向 v 和从 v 流向 u ”是个不错的主意。为了更方便地叙述算法, 先假定图中不存在平行边和反向边。这样就可以用两个邻接矩阵 cap 和 cost 保存各边的容量和费用。为了允许反向增广, 规定 $\text{cap}[v][u]=0$ 并且 $\text{cost}[v][u]=-\text{cost}[u][v]$, 表示沿着 (u, v) 的相反方向增广时, 费用减小 $\text{cost}[u][v]$ 。

限于篇幅, 这里直接给出最小费用路算法。和 Edmonds-Karp 算法类似, 但每次用 Bellman-Ford 算法而非 BFS 找增广路。只要初始流是该流量下的最小费用可行流, 每次增广后的新流都是新流量下的最小费用流。另外, 费用值是可正可负的。在下面的代码中, 为了减小溢出的可能, 总费用 cost 采用 long long 来保存。

```
struct Edge {
    int from, to, cap, flow, cost;
    Edge(int u, int v, int c, int f, int w): from(u), to(v), cap(c), flow(f), cost(w)
    {}
};

struct MCMF {
    int n, m;
    vector<Edge> edges;
    vector<int> G[maxn];
    int inq[maxn]; // 是否在队列中
    int d[maxn]; // Bellman-Ford
    int p[maxn]; // 上一条弧
    int a[maxn]; // 可改进量

    void init(int n) {
        this->n = n;
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap, int cost) {
        edges.push_back(Edge(from, to, cap, 0, cost));
        edges.push_back(Edge(to, from, 0, 0, -cost));
        m = edges.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }

    bool BellmanFord(int s, int t, int& flow, long long& cost) {
        for(int i = 0; i < n; i++) d[i] = INF;
```

```

memset(inq, 0, sizeof(inq));
d[s] = 0; inq[s] = 1; p[s] = 0; a[s] = INF;
queue<int> Q;
Q.push(s);
while(!Q.empty()) {
    int u = Q.front(); Q.pop();
    inq[u] = 0;
    for(int i = 0; i < G[u].size(); i++) {
        Edge& e = edges[G[u][i]];
        if(e.cap > e.flow && d[e.to] > d[u] + e.cost) {
            d[e.to] = d[u] + e.cost;
            p[e.to] = G[u][i];
            a[e.to] = min(a[u], e.cap - e.flow);
            if(!inq[e.to]) { Q.push(e.to); inq[e.to] = 1; }
        }
    }
}
if(d[t] == INF) return false;
flow += a[t];
cost += (long long)d[t] * (long long)a[t];
for(int u = t; u != s; u = edges[p[u]].from) {
    edges[p[u]].flow += a[t];
    edges[p[u]^1].flow -= a[t];
}
return true;
}

//需要保证初始网络中没有负权圈
int MincostMaxflow(int s, int t, long long& cost) {
    int flow = 0; cost = 0;
    while(BellmanFord(s, t, flow, cost));
    return flow;
}
};

```

11.4.5 应用举例

首先需要明确一点: 虽然本节介绍了最大流的 Edmonds-Karp 算法, 但在实践中一般不用这个算法, 而是使用效率更高的 Dinic 算法或者 ISAP 算法。这两个算法虽然也不是很难理解, 但是较之 Edmonds-Karp 来说还是复杂了许多。另一方面, 最小费用流也有更快的算法, 但在实践中一般仍用上述算法, 因为最小费用流的快速算法 (例如网络单纯型法) 大

都很复杂，还没有广泛使用。对此，笔者的建议是：理解 Edmonds-Karp 算法的原理（包括正确性证明），但在比赛中使用 Dinic 或者 ISAP。《算法竞赛入门经典——训练指南》对这两个算法有较为详细介绍，还给出了完整的代码。事实上，读者无须搞清楚它们的原理，只需会使用即可。换句话说，可以把它们当作像 STL 一样的黑盒代码。在算法竞赛中，一般把这样的代码称为模板。

二分图匹配。网络流的一个经典的应用是二分图匹配。在图论中，匹配是指两两没有公共点的边集，而二分图是指：可以把结点集分成两部分 X 和 Y ，使得每条边恰好一个端点在 X ，另一个端点在 Y 。换句话说，可以把结点进行二染色（bicoloring），使得同色结点不相邻。为了方便叙述，在画图时一般把 X 结点和 Y 结点画成左右两列。可以证明：一个图是二分图，当且仅当它不含长度为奇数的圈。

常见的二分图匹配问题有两种。第一种是针对无权图的，需要求出包含边数最多的匹配，即二分图的最大基数匹配（maximum cardinality bipartite matching），如图 11-14（a）所示。

这个问题可以这样求解：增加一个源点 s 和一个汇点 t ，从 s 到所有 X 结点各连一条容量为 1 的弧，再从所有 Y 结点各连一条容量为 1 的弧到 t ，最后把每条边变成一条由 X 指向 Y 的有向弧，容量为 1。只要求出 s 到 t 的最大流，则原图中所有流量为 1 的弧对应了最大基数匹配。

第二种是针对带权图的，需要求出边权之和尽量大的匹配，如图 11-14（b）所示。有些题目要求这个匹配本身是完美匹配（perfect matching），即每个点都被匹配到，而有些题目并不对边的数量做出要求，只要权和最大就可以了。下面先考虑前一种情况，即最大权完美匹配（maximum weighted perfect matching）。

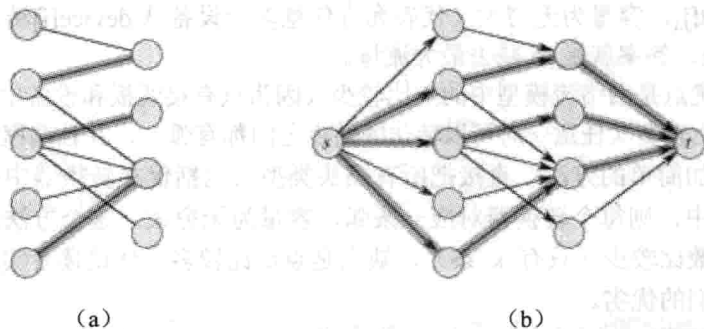


图 11-14 二分图匹配

聪明的读者相信已经找到解决方法了：和最大基数匹配类似，只是原图中所有边的费用为权值的相反数（即前面加一个负号），然后其他边的费用为 0，然后求一个 s 到 t 的最小费用最大流即可。如果从 s 出发的所有弧并不是全部满载（即流量等于容量），则说明完美匹配不存在，问题无解；否则原图中的所有流量为 1 的弧对应最大权完美匹配。

用这样的方法也可以求解第二种情况，即匹配边数没有限制的最大权匹配，只需要在求解 s - t 最小费用流的过程中记录下流量为 0, 1, 2, 3, ... 时的最小费用流，然后加以比较，细节留给读者思考。

例题 11-7 UNIX 插头 (A Plug for UNIX, UVa753)

有 n 个插座, m 个设备和 k ($n, m, k \leq 100$) 种转换器, 每种转换器都有无限多。已知每个插座的类型, 每个设备的插头类型, 以及每种转换器的插座类型和插头类型。插头和插座类型都用不超过 24 个字母表示, 插头只能插到类型名称相同的插座中。

例如, 有 4 个插座, 类型分别为 A, B, C, D; 有 5 个设备, 插头类型分别为 B, C, B, B, X; 还有 3 种转换器, 分别是 $B \rightarrow X$, $X \rightarrow A$ 和 $X \rightarrow D$ 。这里用 $B \rightarrow X$ 表示插座类型为 B, 插头类型为 X, 因此一个插头类型为 B 的设备插上这种转换器之后就“变成”了一个插头类型为 X 的设备。转换器可以级联使用, 例如插头类型为 A 的设备依次接上 $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$ 这 3 个转换器之后会“变成”插头类型为 D 的设备。

要求插的设备尽量多。问最少剩几个不匹配的设备。

【分析】

首先要注意的是: k 个转换器中涉及的插头类型不一定是接线板或者设备中出现过的插头类型。在最坏情况下, 100 个设备, 100 个插座, 100 个转换器最多会出现 400 种插头。当然, 400 种插头的情况肯定是无解的, 但是如果编码不当, 这样的情况可能会让你的程序出现下标越界等运行错误。

笔者第一次尝试本题时使用的方法如下: 转换器有无限多, 所以可以独立计算出每个设备 i 是否可以接上 0 个或多个转换器之后插到第 j 个插座上, 方法是建立有向图 G , 结点表示插头类型, 边表示转换器, 然后使用 Floyd 算法, 计算出任意一种插头类型 a 是否能转化为另一种插头类型 b 。

接下来构造网络: 设设备 i 对应的插头类型编号为 $device[i]$, 插座 i 对应的插头类型编号为 $target[i]$, 则源点 s 到所有 $device[i]$ 连一条弧, 容量为 1, 然后所有 $target[i]$ 到汇点 t 连一条弧, 容量为 1, 对于所有设备 i 和插座 j , 如果 $device[i]$ 可以转化为 $target[j]$, 则从 $device[i]$ 连一条弧到 $target[j]$, 容量为无穷大 (代表允许任意多个设备从 $device[i]$ 转化为 $target[j]$), 最后求 s - t 最大流, 答案就是 m 减去最大流量。

上述算法的优点是网络流模型中的点比较少 (因为只有接线板和设备中出现过的插头类型), 缺点是弧比较多 (任意一对可以转化的结点之间都有弧), 并且编程稍微麻烦一些。

还有一个更加简单的方法: 直接把所有插头类型 (包括仅在转换器中出现的类型) 纳入到网络流模型中, 则每个转换器对应一条弧, 容量为无穷大。这个方法的优点是编程简单, 并且弧的个数比较少 (只有 k 条), 缺点是点数比较多。建议读者实现这两种算法, 然后自行比较它们的优劣。

例题 11-8 矩阵解压 (Matrix Decompressing, UVa 11082)

对于一个 R 行 C 列的正整数矩阵 ($1 \leq R, C \leq 20$), 设 A_i 为前 i 行所有元素之和, B_i 为前 i 列所有元素之和。已知 R, C 和数组 A 和 B , 找一个满足条件的矩阵。矩阵中的元素必须是 $1 \sim 20$ 之间的正整数。输入保证有解。

【分析】

首先根据 A_i 和 B_i 计算出第 i 行的元素之和 A'_i 和第 i 列的元素之和 B'_i 。如果把矩阵里的每个数都减 1, 则每个 A'_i 会减少 C , 而每个 B'_i 会减少 R 。这样一来, 每个元素的范围变成了 $0 \sim 19$, 它的好处很快就能看到。

建立一个二分图, 每行对应一个 X 结点, 每列对应一个 Y 结点, 然后增加源点 s 和汇

点 t 。对于每个结点 X_i , 从 s 到 X_i 连一条弧, 容量为 $A'_i - C$; 从 Y_i 到 t 连一条弧, 容量为 $B'_i - R$ 。而对于每对结点 (X_i, Y_j) , 从 X_i 向 Y_j 连一条弧, 容量为 19。接下来求 $s-t$ 的最大流, 如果所有 s 出发和到达 t 都满载, 说明问题有解, 结点 $X_i \rightarrow Y_j$ 的流量就是格子 (i, j) 减 1 之后的值。

为什么这样做是对的? 请读者思考。

例题 11-9 海军上将 (Admiral, ACM/ICPC NWERC 2012, UVa1658)

给出一个 v ($3 \leq v \leq 1000$) 个点 e ($3 \leq e \leq 10000$) 条边的有向加权图, 求 $1 \sim v$ 的两条不相交 (除了起点和终点外没有公共点) 的路径, 使得权和最小。如图 11-15 所示, 从 1 到 6 的两条最优路径为 1-3-6 (权和为 33) 和 1-2-5-4-6 (权和为 53)。

【分析】

把 2 到 $v-1$ 的每个结点 i 拆成 i 和 i' 两个结点, 中间连一条容量为 1, 费用为 0 的边, 然后求 1 到 v 的流量为 2 的最小费用流即可。

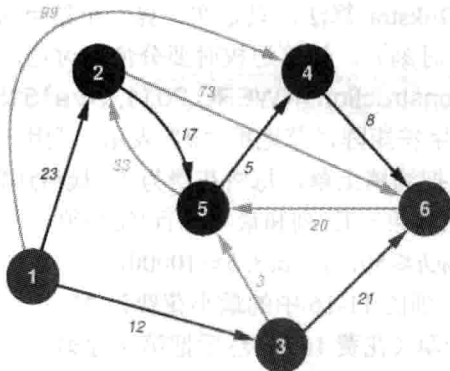


图 11-15 从 1 到 6 的两条最优路径

本题的拆点法是解决结点容量的通用方法, 请读者注意。

例题 11-10 最优巴士路线设计 (Optimal Bus Route Design, ACM/ICPC Taiwan 2005, UVa12264)

给 n 个点 ($n \leq 100$) 的有向带权图, 找若干个有向圈, 每个点恰好属于一个圈。要求权和尽量小。注意即使 (u, v) 和 (v, u) 都存在, 它们的权值也不一定相同。

【分析】

每个点恰好属于一个有向圈, 意味着每个点都有一个唯一的后继。反过来, 只要每个点都有唯一的后继, 每个点一定恰好属于一个圈。“每个东西恰好有唯一的……”让我们想到了二分图匹配。把每个点 i 拆成 X_i 和 Y_i , 原图中的有向边 $u \rightarrow v$ 对应二分图中的边 $X_u \rightarrow Y_v$, 则题目转化为了这个二分图上的最小权完美匹配问题。

11.5 竞赛题目选讲

例题 11-11 有趣的赛车比赛 (Funny Car Racing, UVa 12661)

在一个赛车比赛中, 赛道有 n ($n \leq 300$) 个交叉点和 m ($m \leq 50000$) 条单向道路。有

趣的是：每条路都是周期性关闭的。每条路用 5 个整数 u, v, a, b, t 表示 ($1 \leq u, v \leq n, 1 \leq a, b, t \leq 10^5$)，表示起点是 u ，终点是 v ，通过时间为 t 秒。另外，这条路会打开 a 秒，然后关闭 b 秒，然后再打开 a 秒，依此类推。当比赛开始时，每条道路刚刚打开。你的赛车必须在道路打开的时候进入该道路，并且在它关闭之前离开（进出道路不花时间，所以可以在打开的瞬间进入，关闭的瞬间离开）。

你的任务是从 s 出发，尽早到达目的地 t ($1 \leq s, t \leq n$)。道路的起点和终点不会相同，但是可能有两条道路的起点和终点分别相同。

【分析】

本题是一道最短路问题，但又和普通的最短路问题不太相同：花费的总时间并不是经过的每条边的通过时间之和，还要加上在每个点等待的总时间。还记得第 9 章中的例题“基金管理”吗？该题的决策不仅依赖于状态本身，还依赖于该状态下现金的最大值。本题也是一样：仍然调用标准的 Dijkstra 算法，只是在计算一个结点 u 出发的边权时要考虑 $d[u]$ （即从 s 出发达到 u 的最早时刻）。计算边权时要分情况讨论，细节留给读者思考。

例题 11-12 水塘 (Pool construction, NWERC 2011, UVa1515)

输入一个 h 行 w 列的字符矩阵，草地用“#”表示，洞用“.”表示。你可以把草改成洞，每格花费为 d ，也可以把洞填上草，每格花费为 f 。最后还需要在草和洞之间修围栏，每条边的花费为 b 。整个矩阵第一行/列和最后一行/列必须都是草。求最小花费。 $2 \leq w, h \leq 50, 1 \leq d, f, b \leq 10000$ 。

例如， $d=1, f=8, b=1$ ，则图 11-16 中的最小花费为 27，方法是先把第一行的洞填上草（花费 16），然后把第 3 行第 3 列的草挖成洞（花费 1），再修 10 个单位的围栏。

```

# . . ##
##. ##
#. #. #
#####

```

图 11-16 水塘问题示意图

【分析】

围栏的作用是把草和洞隔开，让人联想到了“割”这个概念。可是“割”只是把图中的结点分成了两个部分，而本题中，草和洞都能有多个连通块。怎么办呢？添加源点 S 和汇点 T ，与其他点相连，则所有本不连通的草地/洞就能通过源点和汇点间接连起来了。

由于草和洞可以相互转换，而且转换还需要费用，所以需要一并在“割”中体现出来。为此，规定与 S 连通的都是草，与 T 连通的都是洞，则 S 需要往所有草格子连一条容量为 d 的边，表示必须把这条弧切断（割的容量增加 d ），这个格子才能“叛逃”到 T 的“阵营”，成为洞。由于题目说明了最外圈的草不能改成洞，从 S 到这些草格子的边容量应为正无穷（在这之前需要把边界上的所有洞填成草，累加出这一步所需的费用）。

同理，所有不在边界上的洞格子往 T 连一条弧，费用为 f ，表示必须把这条弧切断（割的容量增加 f ），才能让这个洞变成草。相邻两个格子 u 和 v 之间需要连两条边 $u \rightarrow v$ 和 $v \rightarrow u$ ，容量均为 b ，表示如果 u 是草， v 是洞，则需要切断弧 $u \rightarrow v$ ；如果 v 是草， u 是洞，则需要切断弧 $v \rightarrow u$ 。

这样，用最大流算法求出最小割，就可以得到本题的最小花费。

例题 11-13 混合图的欧拉回路 (Euler Circuit, UVa10735)

给出一个 V 个点和 E 条边 ($1 \leq V \leq 100, 1 \leq E \leq 500$) 的混合图（即有的边是无向边，有的边是有向边），试求出它的一条欧拉回路，如果没有，输出无解信息。输入保证在忽

略边的方向之后图是连通的。

【分析】

很多混合图问题（例如，混合图的最短路）都可以转化为有向图问题，方法是把无向边拆成两条方向相反的有向边。可惜本题不能使用这种方法，因为本题中的无向边只能经过一次，而拆成两条有向边之后变成了“沿着两个相反方向各经过一次”。所以本题不能拆边，而只能给边定向，就像第 9 章的例题“一个调度问题”那样。

假设输入的原图为 G 。首先把它的无向边任意定向，然后把定向后的有向边单独组成另外一个图 G' 。具体来说，初始时 G' 为空，对于 G 中的每条无向边 $u-v$ ，把它改成有向边 $u \rightarrow v$ ，然后在 G' 中连一条边 $u \rightarrow v$ （注意这个定向是任意的。如果定向为 $v \rightarrow u$ ，则在 G' 中连一条边 $v \rightarrow u$ ）。

接下来检查每个点 i 在 G 中的入度和出度。如果所有点的入度和出度相等，则现在的 G 已经存在欧拉回路。假设一个点的入度为 2，出度为 4，则可以想办法把一条出边变成入边（前提是那条出边原来是无向边，因为无向边才可以任意定向），这样入度和出度就都等于 3 了；一般地，如果一个点的入度为 $\text{in}(i)$ ，出度为 $\text{out}(i)$ ，则只需把出度增加 $(\text{in}(i) - \text{out}(i))/2$ 即可（因为总度数不变，此时入度一定会和出度相等）。如果 $\text{in}(i)$ 和 $\text{out}(i)$ 的奇偶性不同，则问题无解。

如果把 G' 中的一条边 $u \rightarrow v$ 反向成 $v \rightarrow u$ ，则 u 的出度减 1， v 的出度加 1，就像是把一个叫“出度”的物品从结点 u “运输”到了结点 v 。是不是很像网络流？也就是说，满足 $\text{out}(i) > \text{in}(i)$ 的每个点能“提供”一些“出度”，而 $\text{out}(i) < \text{in}(i)$ 的点则“需要”一些“出度”。如果能算出一个网络流，把这些“出度”运输到需要它们的地方，问题就得到了解决（有流量的边对应“把边反向”的操作）。

细节留给读者思考。相信经过了前面题目的锻炼，读者一定可以解决这个问题。

例题 11-14 星际游击队 (Asteroid Rangers, ACM/ICPC World Finals 2012, UVa1279)

三维空间里有 n ($2 \leq n \leq 50$) 个匀速运动的点，第 i 个点的初始坐标为 (x_i, y_i, z_i) ，速度为 (v_x, v_y, v_z) 。求最小生成树会改变多少次。输入保证在任意时刻最小生成树总是唯一的，并且每次变化时，新的最小生成树至少会保持 10^{-6} 个单位时间。

【分析】

不难发现：最小生成树切换的时刻一定对应着某两条边 (u_1, v_1) 和 (u_2, v_2) 的权值相等。一共有 $O(n^2)$ 条边，因此有 $O(n^4)$ 种可能的切换时间（称为事件点）。

最容易想到的做法是把所有可能的事件点按照时间从小到大排序，依次计算每个事件点之后 0.5×10^{-6} 时刻的最小生成树（题目保证了这期间最小生成树不会发生变化），判断它是否和上一个最小生成树相等。假设使用 $O(n^2)$ 时间复杂度的 prim 算法，总时间复杂度为 $O(n^6)$ ，需要优化。

一个行之有效的优化是：假设一个事件点对应 (u_1, v_1) 和 (u_2, v_2) 的权值相等。只有当 (u_1, v_1) 和 (u_2, v_2) 恰好有一个在当前最小生成树，且在该事件点之后这条边会变得比另一条边大时，才有可能发生切换。实践中满足这个条件的事件点非常少，运行效率大幅度提高。

例题 11-15 帮助小罗拉 (Help Little Laura, Beijing 2007, UVa1659)

平面上有 m 条有向线段连接了 n 个点。你从某个点出发顺着有向线段行走，给沿途经

过的每条线段涂一种不同的颜色，最后回到起点。你可以多次行走，给多个回路涂色。可以重复经过一个点，但不能重复经过一条有向线段。如图 11-17 所示是一种涂色方法（虚线表示未涂色）。

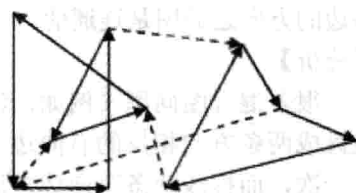


图 11-17 涂色方法示意图

每涂一个单位长度将得到 x 分，但每使用一种颜料将扣掉 y 分。假定颜料有无限多种，如何涂色才能使得分最大？输入保证若存在有向线段 $u \rightarrow v$ ，则不会出现有向线段 $v \rightarrow u$ 。 $n \leq 100$ ， $m \leq 500$ ， $1 \leq x, y \leq 1000$ 。

【分析】

本题的模型是：给出一张有向图，从中选出权和最大的边集，组成若干个有向圈。这里的边权等于题目中的 $dx - y$ ，其中 d 为边的两个端点的欧几里德距离。

由于每个点并不一定只属于一个有向圈，因此例题“最优巴士路线设计”中“匹配后继”的方法不再适用。尽管如此，还是可以建立一个费用流模型：在原图的基础上设每条边的容量为 1，费用为边权，要求找一个流，使得所有结点都满足流量平衡（入流等于出流）条件，且总流量乘以费用的总和最大。这样的模型没有源也没有汇，而且每个结点都要满足流量平衡，所以也没有“最大流”这种说法，称为循环流（circulation）。换句话说，此处要解决的问题是最大费用循环流问题。

对于最大费用流问题，通常会把所有边权取负，变成最小费用流问题。最大费用循环也不例外：把每条边的边权改成 $-dx + y$ ，则问题转化为最小费用循环流问题。这个问题的解决方法 and 最小费用最大流有些类似，只不过每次不是求一条 $s-t$ 的最小费用增广路，而是找整个图的一个负费用增广圈。沿着负费用增广圈进行增广之后，每个结点的流量平衡不会被破坏，而整个循环流的总费用变小了。换句话说，求解最小费用循环流的伪代码就是：

```
while(find_negative_cycle()) augment();
```

根据残量网络的概念不难得出：找负费用增广圈等价于在残量网络中找负权圈——这正是 Bellman-Ford 算法的拿手好戏。

上述算法可以很好地解决本题，但是本题还有一个更有意思的方法，可以避开负圈：新增附加源 s 和附加汇 t ，对于原图中的每条负权边 $u \rightarrow v$ 变成 3 条边： $s \rightarrow v$ ， $v \rightarrow u$ 和 $u \rightarrow t$ ，容量均为 1，但是 $v \rightarrow u$ 的费用为原来的相反数，其他两条边的费用为 0。原图中的正权边 $u \rightarrow v$ 保持不变：容量为 1，费用为权值。

经过这样的处理之后，所有的边都变成正权了，但是网络里出现了很多重边，需要处理一下：对于任意点 u ，假设 $s \rightarrow u$ 的弧有 a 条， $u \rightarrow t$ 的弧有 b 条，则当 $a > b$ 时只保留一条 $s \rightarrow u$ 的弧，容量为 $a - b$ ，删除所有 $u \rightarrow t$ 的弧； $a < b$ 时类似； $a = b$ 时删除所有 $s \rightarrow u$ 和 $u \rightarrow t$ 的弧。处理完毕之后，只需求一次 $s-t$ 最小费用最大流，则求出的最小费用值再加上原图的所有负权之和就是循环流的最小费用值。

是不是很神奇？作为本章最后的“压轴题”，请读者思考这样做的正确性。另外，这种处理负权的方法具有一定的普遍性，有兴趣的读者可以自行研究。

11.6 训练参考

本章的篇幅不长，内容也不多，不过非常重要。考虑到介绍图论算法的书籍和文章很多，本章并没有很正式地介绍各种概念、算法的证明以及严格的复杂度分析，而是把重点放在了程序实现技巧和建模技巧上。本章例题大都不难，建议读者除了掌握不带星号的题目之外也努力弄懂带星号的题目，并且编程实现。例题列表如表 11-1 所示。

表 11-1 例题列表

类 别	题 号	题目名称 (英文)	备 注
例题 11-1	UVa12219	Common Subexpression Elimination	表达式树
例题 11-2	UVa1395	Slim Span	最小生成树
例题 11-3	UVa1151	Buy or Build	最小生成树
例题 11-4	UVa247	Calling Circles	Floyd 算法、连通分量
例题 11-5	UVa10048	Audiophobia	Floyd 算法，最大值最短路
例题 11-6	UVa658	It's not a Bug, it's a Feature!	复杂状态的最短路
例题 11-7	UVa753	A Plug for UNIX	Floyd 算法、二分图最大匹配
例题 11-8	UVa11082	Matrix Decompressing	网络流建模
例题 11-9	UVa1658	Admiral	拆点法，最小费用流
例题 11-10	UVa1349	Optimal Bus Route Design	后继模型；二分图最小权匹配
例题 11-11	UVa12661	Funny Car Racing	特殊图的 Dijkstra 算法
*例题 11-12	UVa1515	Pool construction	最小割模型
*例题 11-13	UVa10735	Euler Circuit	网络流建模
*例题 11-14	UVa1279	Asteroid Rangers	动点的最小生成树
*例题 11-15	UVa1659	Help Little Laura	最小费用循环流

下面是习题。本章的习题大都不难，建议读者至少完成 10 道题目。如果要达到更好的效果，至少需要完成 15 道题目。

习题 11-1 网页跳跃 (Page Hopping, ACM/ICPC World Finals 2000, UVa821)

最近的研究表明，互联网上任何一个网页在平均情况下最多只需要单击 19 次就能到达任意一个其他网页。如果把网页看成一个有向图中的结点，则该图中任意两点间最短距离的平均值为 19。

输入一个 n ($1 \leq n \leq 100$) 个点的有向图，假定任意两点之间都相互到达，求任意两点间最短距离的平均值。输入保证没有自环。

习题 11-2 奶酪里的老鼠 (Say Cheese, ACM/ICPC World Finals 2001, UVa1001)

无限大的奶酪里有 n ($0 \leq n \leq 100$) 个球形的洞。你的任务是帮助小老鼠 A 用最短的时间到达小老鼠 O 所在位置。奶酪里的移动速度为 10 秒一个单位，但是在洞里可以瞬间移动。洞和洞可以相交。输入 n 个球的位置和半径，以及 A 和 O 的坐标，求最短时间。

习题 11-3 因特网带宽 (Internet Bandwidth, ACM/ICPC World Finals 2000, UVa820)

在因特网上，计算机是相互连通的，两台计算机之间可能有多条信息连通路。流通容量是指两台计算机之间单位时间内信息的最大流量。不同路径上的信息流通是可以同时进行的。例如，图 11-18 中有 4 台计算机，总共 5 条路径，每条路径都标有流通容量。从计算机 1 到计算机 4 的流通总容量是 25，因为路径 1-2-4 的容量为 10，路径 1-3-4 的容量为 10，路径 1-2-3-4 的容量为 5。

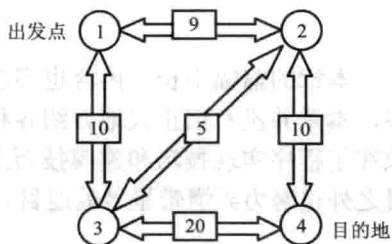


图 11-18 计算机和路径

请编写一个程序，在给出所有计算机之间的路径和路径容量后求出两个给定结点之间的流通总容量（假设路径是双向的，且两方向流动的容量相同）。

习题 11-4 电视网络 (Cable TV Network, ACM/ICPC SEERC 2004, UVa1660)

给定一个 n ($n \leq 50$) 个点的无向图，求它的点连通度，即最少删除多少个点，使得图不连通。如图 11-19 所示，图 11-19 (a) 的点连通度为 3，图 11-19 (b) 的点连通度为 0，图 11-19 (c) 的点连通度为 2（删除 1 和 2 或者 1 和 3）。

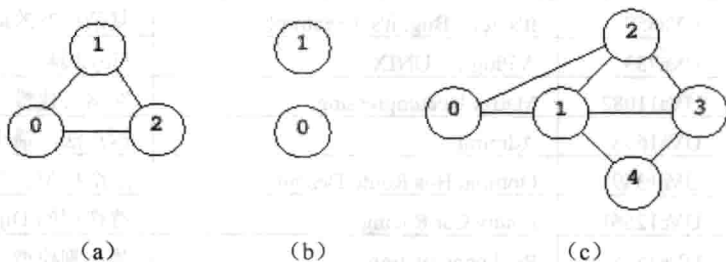


图 11-19 点的连通度

习题 11-5 方程 (Equation, ACM/ICPC NEERC 2007, UVa1661)

输入一个后缀表达式 $f(x)$ ，解方程 $f(x)=0$ 。表达式包含四则运算符，且 x 最多出现一次。保证不会出现除以常数 0 的情况，即至少存在一个 x ，使得 $f(x)$ 不会除 0。所谓后缀表达式，是指把运算符写在运算数的后面。例如， $(4x+2)/2$ 的后缀表达式为 $4x * 2 + 2 /$ 。样例输入与输出如表 11-2 所示。

表 11-2 样例输入与输出

样 例 输 入	样 例 输 出
4 x * 2 + 2 /	x = -1/2
2 2 *	NONE
0 2 x / *	MULTIPLE

习题 11-6 括号 (Brackets Removal, NEERC 2005, UVa1662)

给一个长度为 n 的表达式，包含字母、二元四则运算符和括号，要求去掉尽量多的括号。去括号规则如下：若 A 和 B 是表达式，则 $A+(B)$ 可变为 $A+B$ ， $A-(B)$ 可变为 $A-B'$ ，其中 B' 为 B 把顶层“+”与“-”互换得到；若 A 和 B 为乘法项 (term)，则 $A*(B)$ 变为 $A*B$ ，

$A/(B)$ 变为 A/B' , 其中 B' 为 B 把顶层 “*” 与 “/” 互换得到。本题只能用结合律, 不能用交换律和分配律。

例如, $((a-b)-(c-d)-(z*z*g/f)/(p*(t))*((y-u)))$ 去掉括号以后为 $a-b-c+d-z*z*g/f/p/t*(y-u)$ 。

习题 11-7 电梯换乘 (Lift Hopping, UVa 10801)

在一个假想的大楼里, 有编号为 0~99 的 100 层楼, 还有 n ($n \leq 5$) 座电梯。你的任务是从第 0 楼到达第 k 楼。每个电梯都有一个运行速度, 表示到达一个相邻楼层需要的时间 (单位: 秒)。由于每个电梯不一定每层都停靠, 有时需要从一个电梯换到另一个电梯。换电梯时间总是 1 分钟, 但前提是两座电梯都能停靠在换乘楼层。大楼里没有其他人和你抢电梯, 但你不能使用楼梯 (这是一个假想的大楼, 你无须关心它是否真实存在)。

例如, 有 3 个电梯, 速度分别为 10、50、100, 电梯 1 停靠 0、10、30、40 楼, 电梯 2 停靠 0、20、30 楼, 电梯 3 停靠第 0、20、50 楼, 则从 0 楼到 50 楼至少需要 3920 秒, 方法是坐电梯 1 到达 30 楼 (300 秒), 坐电梯 2 到达 20 楼 (500 秒+换乘 60 秒), 再坐电梯 3 到达 50 楼 (3000 秒+换乘 60 秒), 一共 $300+50+60+3000+60=3920$ 秒。

习题 11-8 净化器 (Purifying Machine, ACM/ICPC Beijing 2005, UVa1663)

给 m 个长度为 n 的模板串。每个模板串包含字符 0,1 和最多一个星号 “*”, 其中星号可以匹配 0 或 1。例如, 模板 01* 可以匹配 010 和 011 两个串, 而模板集合 {*01, 100, 011} 可以匹配串 {001, 101, 100, 011}。

你的任务是改写这个模板集合, 使得模板的个数最少。例如, 上述模板集合 {*01, 100, 011} 可以改写成 {0*1, 10*}, 匹配到的字符串集合仍然是 {001, 101, 100, 011}。 $n \leq 10$, $m \leq 1000$ 。

习题 11-9 机器人警卫 (Sentry Robots, ACM/ICPC SWERC 2012, UVa12549)

在一个 Y 行 X 列 ($1 \leq Y, X \leq 100$) 的网格里有空地 (.), 重要位置 (*) 和障碍物 (#), 如图 11-20 所示。用最少的机器人看守所有重要位置。每个机器人要放在一个格子里, 面朝上下左右 4 个方向之一。机器人会发出激光, 一直射到障碍物为止, 沿途都是看守范围。机器人不会阻挡射线, 但不同的机器人不能放在同一个格子。

Grid	Solution
<pre> * * . . . * # * . . # * * . . </pre>	<pre> . → * . . . ↑ # ↑ . . # ↓ * . . </pre>

图 11-20 “机器人警卫”问题示意图

习题 11-10 Risk 游戏 (Risk, NWERC 2010, UVa12011)

有 n ($n \leq 100$) 个阵地。已知我方在每个阵地上的士兵数 (0~100 的整数), 其中士兵大于 0 表示该阵地由我方占领, 否则为敌方占领。对于一个我方阵地, 如果其相邻的阵地中有敌方阵地, 则称为边界阵地 (border region)。

现在对我方士兵进行调动 (每次可以把一个士兵从一个阵地移动到相邻的我方阵地, 操作可以进行任意多次), 在保证我方不丢失阵地的情况下 (即我方每个阵地上的人数不为 0), 使得我方的边界阵地中人数最少的阵地的人数尽量多。

输入保证我方至少有一个阵地, 敌方也至少有一个阵地, 且至少有一个我方阵地与敌

方阵地相邻。

习题 11-11 占领新区域 (Conquer a New Region, ACM/ICPC Changchun 2012, UVa1664)

n ($n \leq 200000$) 个城市形成一棵树，每条边有权值 $C(i,j)$ 。任意两个点的容量 $S(i,j)$ 定义为 i 与 j 唯一通路上的容量的最小值。找一个点（它将成为中心城市），使得它到其他所有点的容量之和最大。

习题 11-12 岛屿 (Islands, ACM/ICPC CERC 2009, UVa1665)

输入一个 $n*m$ 矩阵，每个格子里都有一个 $[1, 10^9]$ 正整数。再输入 T 个整数 t_i ($0 \leq t_1 \leq t_2 \leq \dots \leq t_T \leq 10^9$)，对于每个 t_i ，输出大于 t_i 的正整数组成多少个四连块。如图 11-21 所示，大于 1 的正整数组成两块，大于 2 的组成 3 块。

评论：这个题目虽然和图论没什么关系，但是可以用到本章介绍的某个数据结构。

1	2	3	3	1
1	3	2	2	1
2	1	3	4	3
1	2	2	2	2

1	2	3	3	1
1	3	2	2	1
2	1	3	4	3
1	2	2	2	2

图 11-21 “岛屿”问题示意图

习题 11-13 最短路线 (Walk, ACM/ICPC Jinhua 2012, UVa1666)

平面上有 n ($n \leq 50$) 个建筑物，求从 (x_1, y_1) 到 (x_2, y_2) 的一条路，使得转弯次数最少。建筑物都是坐标平行于坐标轴的矩形，可以相互接触但不会重叠（接触的点或者边都不能通过）。你只能沿着平行于坐标轴的直线走，可以沿着建筑物的边走，但不能穿过建筑物。无解输出 -1。

提示：本题在细节上容易出错。

习题 11-14 乱糟糟的网络 (Network Mess, ACM/ICPC Tokyo 2005, UVa1667)

有一棵 n ($n \leq 50$) 个叶子的无权树。输入两两叶子的距离，恢复出这棵树并输出每个非叶子结点的度数。

习题 11-15 绿色行动 (Let's Go Green, ACM/ICPC Jakarta 2012, UVa1668)

输入一棵 n ($2 \leq n \leq 100000$) 个结点的树，每条边上都有一个权值。要求用最少的路径覆盖这些边，使得每条边被覆盖的次数等于它的权值，如图 11-22 所示。

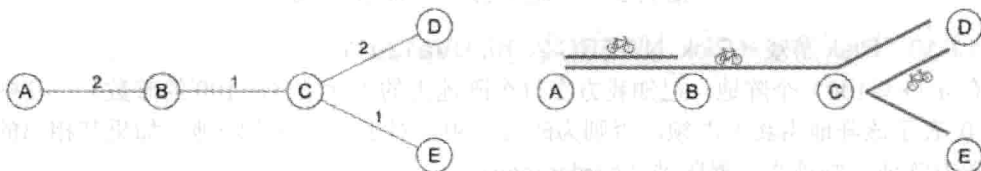


图 11-22 “绿色行动”问题示意图

习题 11-16 交换房子 (Holiday's Accomodation, ACM/ICPC Chengdu 2011, UVa1669)

有一棵 n ($2 \leq n \leq 10^5$) 个结点的树，每个结点住着一个人。这些人想交换房子（即每

个人都要去另外一个人的房子，并且不同人不能去同一个房子）。要求安排每个人的行程，使得所有人旅行的路程长度之和最大。

习题 11-17 王国的道路图 (Kingdom Roadmap, ACM/ICPC NEERC 2011, UVa1670)

输入一个 n ($n \leq 100000$) 个结点的树，添加尽量少的边，使得任意删除一条边之后图仍然连通。如图 11-23 所示，最优方案用虚线表示。

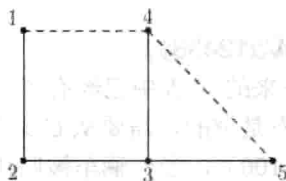


图 11-23 “王国的道路图”问题示意图

习题 11-18 交通堵塞 (Traffic Jam, ACM/ICPC Dhaka 2009, UVa12214)

有一条包含 n ($1 \leq n \leq 25$) 段的单向折线，你想开着一辆会“飞”的车从折线起点到折线终点，且耗油量最少。沿着折线方向正常行驶时单位耗油量为 1，“飞行”时单位耗油量为 f ($2 \leq f \leq 5$)。如图 11-24 所示， $f=2$ ，折线为 $(0,0)-(2,2)-(2,-2)$ ，沿折线行驶的耗油量为 $2.828+4=6.828$ ，最优解是从 $(0,0)$ “飞”到 $(2,-1.154)$ ，然后正常行驶到 $(2,-2)$ ，耗油量为 $2 \times 2.309 + 0.846 = 5.464$ 。

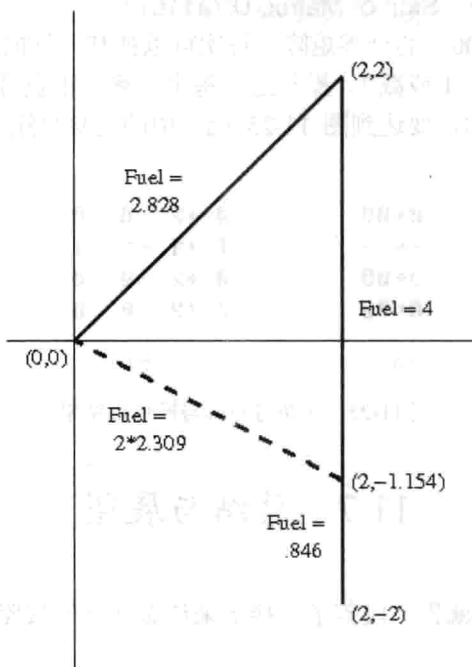


图 11-24 行驶路线

习题 11-19 火车延误 (Train Delays, NWERC 2011, UVa1518)

有 n ($1 \leq n \leq 100$) 条火车线路，均为每小时发车一次。输入每条线路的起点站和终点

站名称、发车时间 m ($0 \leq m \leq 59$)、正点运行时间 t ($1 \leq t \leq 300$)、到达时间、延误概率百分比 p ($0 \leq p \leq 100$) 和最大延误时间 d ($1 \leq d \leq 120$)。如果火车延误，实际延误时间为 $[1, d]$ 内均匀分布的整数，并且只有在列车发车之后才能知道是否会延误（但此时已经无法换车）。

假定换乘不花时间（即使到达时刻等于要换乘的列车的发车时刻，也可以完成换乘），并且可以根据实际延误情况动态改变乘车计划，你的任务是让总时间的期望值最小。出发时间可以自己定。

习题 11-20 租车 (Rent a Car, UVa12433)

你想经营一家租车公司。接下来的 N 天中已经有了一些订单，其中第 i 天需要 r_i 辆车 ($0 \leq r_i \leq 100$)。初始时，你的仓库是空的，需要从 C 家汽车公司里买车，其中第 i 家公司里有 c_i 辆车，单价是 p_i ($1 \leq c_i, p_i \leq 100$)。当一辆车被归还给租车公司之后，你必须把它送去保养之后才能再次租出去。一共有 R 家服务中心，其中第 i 家保养一次需要 d_i 天，每辆车的费用为 s_i ($1 \leq d_i, s_i \leq 100$)。这些服务中心都很大，可以接受任意多辆车同时保养。你的仓库很大，可以容纳任意多辆车。你的任务是用最小的费用满足所有订单。 $1 \leq N, C, R \leq 50$ 。

例如， $N=3, C=2, R=1, r=\{10, 20, 30\}, c_1=40, p_1=90, c_2=15, p_2=100, d_1=1, s_1=5$ ，最优方案是：先买 50 辆车，其中在公司 1 买 40 辆，公司 2 买 10 辆，费用为 $90 \times 40 + 100 \times 10 = 4600$ 。第一天白天租出去 10 辆车，晚上收回之后送到服务中心保养一天，费用为 $5 \times 10 = 50$ ，第 3 天白天可以再次出租。第 2 天出租 20 辆车，第 3 天把剩下的 20 辆车和保养后的 10 辆车一起出租。总费用为 $4600 + 50 = 4650$ 。

习题 11-21 矩阵中的符号 (Sign of Matrix, UVa11671)

有一个 $n \times n$ ($2 \leq n \leq 100$) 的全零矩阵，每次可以把某一行的所有元素加 1 或减 1，也可以把某一列的所有元素加 1 或减 1。操作之后每个元素的正负号已知，问：至少需要多少次操作？无解输出 -1。例如，要达到图 11-25 (a) 中的正负号矩阵，至少需要 3 次操作，如图 11-25 (b) 所示。

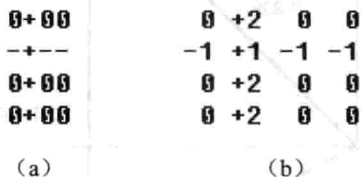


图 11-25 正负号矩阵与操作后结果

11.7 总结与展望

至此，前 11 章的讲解就告一段落了。接下来该做什么？按照先后顺序，建议读者做 3 件事：

巩固前 11 章的内容。先别急，在继续前进之前，笔者建议大家先把前 11 章的内容学扎实。什么叫“学扎实”？每章的“小结和习题”部分都有具体描述，这里不再赘述。

但是有一点需要注意：理解一个题解和自己独立推导出所有细节还是不一样的，所以在看完一个难题的题解之后最好把它做两遍：一遍是刚看完题解以后“趁热打铁”，一遍

是等忘掉题解后自己从头推导一遍。

学习《算法竞赛入门经典——训练指南》。确保前 11 章基础扎实之后，推荐学习《算法竞赛入门经典——训练指南》。该书主要是讲解本书前 11 章中没有涉及的知识点，如表 11-3 所示。

表 11-3 《算法竞赛入门经典——训练指南》知识点介绍

内 容	名 称	知 识 点
第 1 章	算法设计基础	Floyd 判圈算法、扫描法、降维法、LIS 的 $O(n \log n)$ 算法、四边形不等式、Joseph 问题的递推解法
第 2 章	数学基础	剩余系和乘法逆、中国剩余定理、离散对数、Nim 游戏和 Sprague-Grundy 定理、马尔科夫过程、置换分解成循环、Burnside 引理、Polya 定理、高斯消元、高斯-约当消元、矩阵的秩、Q 矩阵和快速矩阵幂、三分法求凸函数极值、自适应辛普森公式
第 3 章	实用数据结构	ADT、树状数组 (BIT)、RMQ 问题、线段树、Trie、KMP、Aho-Corasick 自动机、后缀数组及 LCP、Hash 方法、Treap 和伸展树，以及用它们实现的名次树和可分裂合并的序列
第 4 章	几何问题	基本向量几何、点和直线的关系、多边形的面积、与圆和球相关的计算、点在多边形内判定、凸包、旋转卡 (qia) 壳、半平面交、PSLG、三维几何基础、三维凸包
第 5 章	图论算法与模型	DFS 应用：无向图的割顶和桥、无向图的双连通分量、有向图的强连通分量、2-SAT 问题、差分约束系统、最小瓶颈路问题、次小生成树问题、最小有向生成树 (树形图)、LCA 问题、Kuhn-Munkres 算法、稳定婚姻问题、二分图最大匹配的应用 (最小覆盖、最大独立集、DAG 最小路径覆盖)、Dinic 算法和 ISAP 算法、网络流模型变换技巧 (多源多汇、下界、循环流、流量不固定的费用流) 和经典应用 (最大闭合子图、最大密度子图等)
第 6 章	更多算法专题	轮廓线动态规划 (包括带连通信息的)、嵌套数据结构 (二维线段树等)、分块数据结构、minimax 搜索和 alpha-beta 剪枝、舞蹈链和 DLX 算法、二维和三维仿射变换及其矩阵、离散化、几何扫描法 (包括 BST 的使用)、运动规划、Pick 定理、Lucas 定理、高次模方程和原根、多项式乘法与 FFT、线性规划

学习本书第 12 章。有了前 11 章和《算法竞赛入门经典——训练指南》的基础，现在可以去“啃”第 12 章了。说“啃”，是因为这一章的内容实际上已经不属于入门的范畴，而是一些高级内容，甚至还包括一些世界顶级比赛的压轴题。这样的安排是有意的，因为本书的目的并不仅仅是让读者入门，而是“从入门开始一直伴随读者”。正如第 2 版前言所说，请把这一章看作是游戏通关之后多出来的 Hard 模式。

难题主要分为 3 种。一是需要“生僻知识”的，二是思维难度大的，三是编程实现复杂的。本书第 12 章在这 3 种难题中精选了一些值得学习的题目，顺便讲解了相关知识点和解题方法，包括 DFA、NFA 和正规表达式、DAWG、树的分治、欧拉序列、轻重路径剖分 (树链剖分)、LCA 转 RMQ、Link-Cut 树、可持久化数据结构、多边形的布尔运算和偏移、非完美算法等。

准备好了吗？让我们开始迎接真正的挑战吧！